

Project 4 (in Java): Huffman-coding. You will be given a file contains a list of <chr, prob> pairs (see input format below and data1.) For this project, you are to implement the first part of Huffman coding scheme that i) construct an ordered linked list based the input data; ii) construct a binary tree from the linked list; and iii) produce the code table.

You will be given 2 data files: data1, and data2; data1 is identical to the data using in class and data2 contains a larger <chr, prob> pairs. Note: probabilities given in the input pairs have been multiplied by 100.

What you need to do as follows:

- 1) Using data1, draw an illustration as taught in class the process of constructing Huffman coding scheme as follows:
 - i) Using linked list insertion sort to construct an ordered linked-list based on the input pairs.
 - ii) construct the binary tree from the sorted linked list;
 - iii) using the binary tree to construct the code table;
 - iv) Then, write the pre-order, in-order, and post-order traversal on the binary tree.
- 2) Implement and debug your program according to the specs given below until it passes compilation.
- 3) Run and debug your program with data1 until your program produces the same result as in your illustration in 1).
- 4) Then, run your program with data2.

Include in your hard copy (word file):

- Cover page
- Your illustration drawings. (-2 pts without the illustration!!)
- source code
- data1 // with caption ***Below is data1***
- outFile1 on data1 // with caption *** Below is outFile1 from data1***
- logFile on data1 // with caption *** Below is logFile from data1***// print up to 3 pages.
- data2 // with caption ***Below is data2 ***
- outFile1 on data2 // with caption *** Below is outFile1 from data2***
- logFile on data2 // with caption *** Below is logFile from data2 *** // print up to 3 pages.

Language: Java

Project Name: Huffman coding

Project points: 10 pts

Due Date: Soft copy (*.zip) and hard copies :

- 0 (10/10 pts): on time, 3/8/2025 Sunday before midnight, 11:59pm.
- (-10/10 pts): non-submission, 3/8/2025 Sunday after midnight

*** All submission MUST include Soft copy (*.zip) and hard copy (*.docx) in the same email attachments with correct email subject as stated in the project submission requirement; otherwise, your submission will be rejected.

Email subject: (323) firstName lastName <Project 4: Huffman coding (Java)>

*** Inside the email body includes:

- Your answer to the five questions (-2 if omitted)
- Screen recoding link. (-2 without the recording!)

*** Place your screen recording link in your project submission email body below the 5 questions.

I. Inputs:

- a) inFile (args [0]): A file contains a list of <chr prob> pairs with the following format:
chr₁ prob₁ // data type for chr₁ is string and data type for prob₁ is integer which has been multiply by 100.
chr₂ prob₂
:
:
chr_n prob_n

II. Outputs:

- a) outFile1 (arg1 [1]): As program dictates.
- b) logFile (arg2 [2]): As program dictates.

III. Data structure:

- A HtreeNode class

- (string) chStr
- (int) prob
- (string) code
- (HtreeNode) next
- (HtreeNode) left
- (HtreeNode) right

Method:

- constructor (chStr, prob, code, next, left, right) // Creates a HtreeNode with the given data.
- printNode (T, fileOut) // given a HtreeNode, T, print T's chStr, T's prob, T's code
// For example, if T's chStr is "DE", T's prob is 15, T's code is "011", then print
("DE", 15, "011") →

- A HuffmanCoding class

- (HtreeNode) listHead
- (HtreeNode) Root

Methods:

- constructor(s) // do whatever you deem fit.
- (HtreeNode) findSpot (...) // re-use code from your previous projects.
- listInsert (Spot, newNode)
// newNode.next ← Spot.next
// Spot.next ← newNode
- (HtreeNode) constructHuffmanLList (...) // **Algorithm is given below.**
- printList (listHead, fileOut) // Call printNode (...) to print each node in the list, including the dummy node;
//Note: fileOut maybe outFile1 or logFile.
- (HtreeNode) constructHuffmanBinTree (...) // **Algorithm is given below.**
- (bool) isLeaf (node) // returns true if a given node is a leaf, i.e., its left and right are null.
- preOrderTraversal (...) // **Algorithm is given below.**
- inOrderTraversal (...) // on your own.
- postOrderTraversal (...) // on your own.
- constructCharCode (...) // **Algorithm is given below**

IV. Main (...)

Step 0: check argc count is correct; check files, one-by-one, if each file can be opened.

inFile, outFile1, logFile ← open via argv []

Step 1: listHead ← constructHuffmanLList (inFile, logFile)

Step 2: outFile1 ← "In main (): printing list after list is constructed"
printList (listHead, outFile1)

Step 3: Root ← constructHuffmanBinTree (listHead, logFile)

Step 4: outFile1 ← "In main (): printing character codes"
constructCharCode (Root, "", outFile1) // "" is an empty string

Step 5: outFile1 ← "In main (): printing pre-Order traversal of the tree"
preOrderTraversal (Root, outFile1)

Step 6: outFile1 ← "In main (): printing in-Order traversal of the tree"
inOrderTraversal (Root, outFile1)

Step 7: outFile1 ← "In main (): printing post-Order traversal of the tree"
postOrderTraversal (Root, outFile1)

Step 8: close all files

V. (HtreeNode) constructHuffmanLList (inFile, logFile)

Step 0: logFile $\leftarrow$ “*** entering constructHuffmanLList () method”
 Step 1: listHead $\leftarrow$ get a HtreeNode (“dummy”, 0, ‘’, null, null, null) as the dummy node // use constructor
 Step 2: chr $\leftarrow$ get from inFile
 prob $\leftarrow$ get from inFile
 logFile $\leftarrow$ “*** In constructHuffmanLList (), chr = prob=” // print the value.
 Step 3: newNode $\leftarrow$ get a new HtreeNode (chr, prob, “”, null, null, null) // “” is an empty string
 logFile $\leftarrow$ “*** In constructHuffmanLList (), printing newNode” // print the value.
 printNode (newNode, logFile)
 Step 4: Spot $\leftarrow$ findspot (listHead, newNode)
 Step 5: listInsert (Spot, newNode)
 Step 6: logFile $\leftarrow$ “*** In constructHuffmanLList () printing list after inserting a newNode”
 printList (listHead, logFile)
 Step 7: repeat step 2 – step 6 until the end of inFile
 Step 8: “*** entering constructHuffmanLList () method and returning listHead”
 Step 9: return listHead

VI. (HtreeNode) constructHuffmanBinTree (listHead, logFile)

Step 0: logFile $\leftarrow$ “*** Entering constructHuffmanBinTree () method”
 Step 1: newNode $\leftarrow$ get a new HtreeNode (“”, 0, “”, null, null, null)
 newNode’s prob $\leftarrow$ the sum of prob of the first and second node of the list // first is the node after dummy
 newNode’s chStr $\leftarrow$ concatenate chStr of the first node and chStr of the second node in the list
 newNode’s left $\leftarrow$ the first node after dummy node
 newNode’s right $\leftarrow$ the second node after dummy node
 newNode’s next $\leftarrow$ null
 Step 2: logFile $\leftarrow$ “*** In constructHuffmanBinTree, printing newNode”
 printNode (newNode, logFile)
 Step 2: Spot $\leftarrow$ findspot (listHead, newNode)
 Step 3: listInsert (Spot, newNode)
 Step 4: listHead’s next $\leftarrow$ the third node after dummy node // listHead’s next next next
 Step 5: logFile $\leftarrow$ “*** In constructHuffmanBinTree method, printing the list after inserting newNode”
 printList (listHead, logFile)
 Step 6: repeat step 1 – step 4 until the list only has one node left after the dummy node
 Step 7: return listHead’s next

VII. constructCharCode (T, code, outFile1)

 if isLeaf (T)
 T’s code $\leftarrow$ code
 outFile1 $\leftarrow$ output T’s chStr and T’s code to outFile1 // space between the two and one per text line
 else
 constructCharCode (T’s left, code + “0”, outFile1) //string concatenation
 constructCharCode (T’s right, code + “1”, outFile1) //string concatenation

X. preOrderTraval (T, outFile1) // In recursion

 if isLeaf (T)
 printNode (T, outFile1) // output to outFile1, see printing format in treeNode class in above
 else
 printNode (T, outFile1)
 preOrderTraval (T’s left, outFile1)
 preOrderTraval (T’s right, outFile1)